



บทที่ 5 การประเมินราคาและความพยายาม (Cost and Effort Estimates)

ผศ.วาสนา เหง้าเกษ



หัวข้อ

1. แบบจำลองต้นทุนเชิงสร้างสรรค์ (Constructive Cost Model - COCOMO)
2. การวิเคราะห์ฟังก์ชันพอยต์ (Function Point Analysis - FPA)

1. แบบจำลองต้นทุนเชิงสร้างสรรค์ (Constructive Cost Model - COCOMO)

การประมาณการต้นทุนและทรัพยากรที่จำเป็นสำหรับโครงการซอฟต์แวร์จำเป็นในการบริหารจัดการโครงการ แบบจำลองการประเมินเชิงอัลกอริทึม (Algorithmic Cost Models) ได้รับการพัฒนาขึ้นเพื่อการประเมินที่แม่นยำ โดยใช้หลักการทางคณิตศาสตร์และสถิติเพื่อสร้างสมการที่สามารถทำนายค่าใช้จ่าย แรงงาน (Effort) และระยะเวลาในการพัฒนาโครงการซอฟต์แวร์ได้ โดยอาศัยข้อมูลเชิงปริมาณจากโครงการในอดีตมาเป็นพื้นฐานในการนิยามค่าคงที่และตัวแปรต่าง ๆ

แบบจำลอง COCOMO หรือ Constructive Cost Model ได้รับการพัฒนาขึ้นโดย ดร. บาร์รี โบห์ม (Dr. Barry Boehm) ในช่วงปลายทศวรรษ 1970 เพื่อตอบสนองความต้องการที่เพิ่มขึ้นในการประมาณการต้นทุน เวลา และทรัพยากรสำหรับโครงการซอฟต์แวร์อย่างมีแบบแผนและเป็นระบบ แนวคิดพื้นฐานของ COCOMO คือการใช้สมการทางคณิตศาสตร์เพื่อเชื่อมโยงขนาดของซอฟต์แวร์เข้ากับแรงงานและเวลาในการพัฒนาโดยขนาดของซอฟต์แวร์จะถูกวัดในรูปของจำนวนพันบรรทัดของโค้ด (Kilo Lines of Code - KLOC)

COCOMO เป็นโมเดลทางคณิตศาสตร์ที่ใช้ ประมาณการต้นทุน ระยะเวลา และจำนวนคน ที่ต้องใช้ในการ พัฒนาซอฟต์แวร์ หลักการพื้นฐานคือ ซอฟต์แวร์ขนาดใหญ่ (วัดจากจำนวนบรรทัดของโค้ด หรือ LOC - Lines of Code) จะใช้แรงงานและเวลามากขึ้นตามความสัมพันธ์แบบ Exponential

COCOMO I เกิดขึ้นในปี ค.ศ. 1981 และได้กลายเป็นเครื่องมือที่ถูกนำไปใช้กันอย่างแพร่หลายในวงการ วิศวกรรมซอฟต์แวร์ COCOMO I ถูกออกแบบมาสำหรับกระบวนการพัฒนาแบบลำดับขั้น (Waterfall model)

COCOMO II ในปี ค.ศ. 2000 เป็นวิวัฒนาการที่สำคัญซึ่งสะท้อนให้เห็นว่าอุตสาหกรรมซอฟต์แวร์มีความ ซับซ้อนมากขึ้นการประมาณการจึงไม่สามารถอาศัยเพียงแค่ขนาดของโค้ดอีกต่อไป แต่ต้องรวมถึงปัจจัยเชิง คุณภาพและสิ่งแวดล้อมที่ส่งผลต่อการพัฒนาด้วย วิวัฒนาการนี้แสดงให้เห็นว่าแบบจำลองการประเมินต้นทุนนั้น จำเป็นต้องมีความยืดหยุ่นและปรับตัวได้ตามแนวปฏิบัติในการพัฒนาที่หลากหลายมากขึ้น เช่น การพัฒนาแบบ Agile ที่มีการทำงานเป็นรอบและเน้นความยืดหยุ่น

การจำแนกประเภทโครงการ

ประเภทของโครงการซอฟต์แวร์แบ่งเป็น 3 ประเภทหลัก โดยพิจารณาจากคุณลักษณะของซอฟต์แวร์ คุณสมบัติของทีมพัฒนา และสภาพแวดล้อมที่ใช้ในการพัฒนา การจำแนกนี้ส่งผลโดยตรงต่อค่าคงที่ทางคณิตศาสตร์ที่ใช้ในสมการ ซึ่งจะแตกต่างกันไปตามความซับซ้อนของแต่ละประเภทโครงการ

1. **โครงการประเภท Organic** โครงการประเภทนี้โดยทั่วไปจะมีขนาดเล็ก มีความซับซ้อนต่ำ และมักพัฒนาโดยทีมงานขนาดเล็กที่มีประสิทธิภาพสูงในด้านแอปพลิเคชันและสภาพแวดล้อมการทำงานที่เกี่ยวข้อง โครงการประเภท Organic มักมีข้อกำหนดที่ไม่เข้มงวดมากนักและมีการสื่อสารภายในทีมที่มีประสิทธิภาพสูง เนื่องจากสมาชิกทีมมีความคุ้นเคยกับปัญหาที่ต้องแก้ไขเป็นอย่างดี ตัวอย่างที่ชัดเจนได้แก่ ระบบจัดการคลังสินค้าขนาดเล็ก

2. **โครงการประเภท Semi-detached** โครงการกลุ่มนี้มีความซับซ้อนและมีขนาดที่อยู่ระหว่าง Organic และ Embedded ทีมพัฒนาประกอบด้วยบุคลากรที่มีประสบการณ์หลากหลาย ตั้งแต่ผู้เชี่ยวชาญไปจนถึงผู้ที่ไม่มีความรู้ นอกจากนี้ โครงการยังมีความผสมผสานระหว่างข้อกำหนดที่เข้มงวดและไม่เข้มงวด ตัวอย่างของโครงการประเภทนี้คือระบบจัดการฐานข้อมูล
3. **โครงการประเภท Embedded** เป็นโครงการที่มีความซับซ้อนสูงที่สุด โดยซอฟต์แวร์จะถูกพัฒนาภายใต้ข้อกำหนดที่เข้มงวดและตายตัวทั้งในด้านฮาร์ดแวร์ ซอฟต์แวร์ และการดำเนินงาน การพัฒนาต้องอาศัยผู้เชี่ยวชาญที่มีทักษะสูง และบางครั้งต้องใช้ทีมงานขนาดใหญ่ เนื่องจากข้อกำหนดมีความเข้มงวดสูง จึงมักเกี่ยวข้องกับข้อบังคับหรือกฎระเบียบที่จำเป็นต้องปฏิบัติตามอย่างเคร่งครัด ตัวอย่างได้แก่ ระบบควบคุมการบิน หรือระบบตู้ ATM



ค่าคงที่ a, b, c, และ d ที่ใช้ในสมการ COCOMO จะถูกกำหนดตามการจำแนกประเภทโครงการเหล่านี้ ซึ่งสะท้อนถึงความสัมพันธ์ที่ไม่เป็นเชิงเส้นระหว่างขนาดของซอฟต์แวร์และ Effort ในการพัฒนา

โดยโครงการที่มีความซับซ้อนสูงกว่า (Embedded) จะมีค่าตัวคูณ Effort ที่สูงขึ้นอย่างมีนัยสำคัญ เมื่อเทียบกับโครงการที่มีความซับซ้อนต่ำกว่า (Organic)

1. Basic COCOMO
2. Intermediate COCOMO
3. Detailed COCOMO

1. Basic COCOMO

Basic COCOMO เป็นแบบจำลองที่เรียบง่ายที่สุดในตระกูล COCOMO เหมาะสำหรับการประมาณการเบื้องต้นอย่างรวดเร็วในช่วงเริ่มต้นของโครงการ แบบจำลองนี้มีข้อสมมติฐานว่า Effort ในการพัฒนาขึ้นอยู่กับขนาดของซอฟต์แวร์เพียงอย่างเดียว ซึ่งวัดในหน่วย KLOC

$$Effort(E) = a \times (KLOC)^b$$

$$Time(T) = c \times (Effort)^d$$

$$Person(P) = E/T$$

KLOC (Kilo Lines of Code): จำนวนบรรทัดของโค้ด (หน่วยเป็นพันบรรทัด)

Effort (E): แรงงานที่ต้องใช้ (หน่วยเป็น Person-Months หรือ คน-เดือน)

Time (T): ระยะเวลาพัฒนา (หน่วยเป็น Months หรือ เดือน)

Person (P): จำนวนคนที่เหมาะสม (คน)

(ค่าคงที่ a , b , c , d จะเปลี่ยนไปตามประเภทโครงการในข้อ 1)

ประเภทโครงการ (Mode)	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32



ตัวอย่างการคำนวณ (Example)

สมมติว่าเราได้รับโจทย์ให้ทำ ระบบห้องสมุด (Organic Mode) ซึ่งประเมินแล้วว่าน่าจะมีโค้ดประมาณ 32,000 บรรทัด (32 KLOC)

1. หา Effort (แรงงานที่ต้องใช้):

$$E = 2.4 \times (32)^{1.05} \approx 91 \text{ Person-Months}$$

(แปลว่าถ้าใช้คนทำงาน 1 คน ต้องทำ 91 เดือนถึงเสร็จ)

2. หา Time (ระยะเวลาที่ใช้):

$$T = 2.5 \times (91)^{0.38} \approx 14 \text{ Months}$$

(โครงการนี้ควรใช้เวลาประมาณ 1 ปี 2 เดือน)

3. หา Person (จำนวนคนที่เหมาะสม):

$$P = 91/14 \approx 6.5 \text{ People}$$

(ควรจัดทีมประมาณ 6-7 คน)

2. Intermediate COCOMO

เป็นการขยายขอบเขตและปรับปรุงความแม่นยำของ Basic COCOMO แบบจำลองนี้ตระหนักว่า Effort ในการพัฒนาไม่ได้ขึ้นอยู่กับขนาดของโค้ดเพียงอย่างเดียว แต่ยังได้รับอิทธิพลจากปัจจัยอื่นอีก 15 ปัจจัย หรือที่เรียกว่า **ปัจจัยต้นทุน (Cost Drivers)** แบ่งเป็น 4 หมวดหมู่หลัก

- 1) คุณลักษณะของผลิตภัณฑ์ (Product Attributes) ประเมินความต้องการด้านความน่าเชื่อถือ ขนาดฐานข้อมูล และความซับซ้อนของซอฟต์แวร์
- 2) คุณลักษณะของฮาร์ดแวร์ (Hardware Attributes) พิจารณาข้อจำกัดด้านประสิทธิภาพขณะรัน ข้อจำกัดด้านหน่วยความจำ และเวลาการตอบสนองที่ต้องการ
- 3) คุณลักษณะของบุคลากร (Personnel Attributes) ประเมินความสามารถและประสบการณ์ของบุคลากรในด้านต่าง ๆ
- 4) คุณลักษณะของโครงการ (Project Attributes) พิจารณาการใช้เครื่องมือซอฟต์แวร์ การประยุกต์ใช้วิธีการทางวิศวกรรมซอฟต์แวร์ และข้อกำหนดด้านตารางเวลา

Cost Drivers หรือ ตัวขับเคลื่อนต้นทุน ช่วยปรับค่า เพราะในโลกความเป็นจริง โปรเจกต์ขนาดเท่ากัน ไม่ได้แปลว่าจะใช้แรงงานเท่ากันเสมอไป (เช่น ทีมเก่งกับทีมไม่เก่ง หรือ งานซับซ้อนกับงานง่าย)

สูตรของ Intermediate COCOMO

สิ่งที่เปลี่ยนไปจาก Basic มีจุดเดียวคือการคูณ **EAF** เข้าไปครับ:

$$Effort(E) = a \times (KLOC)^b \times EAF$$

- a, b : ค่าคงที่ (ตารางจะต่างจาก Basic เล็กน้อย)
- **KLOC**: ขนาดโค้ด (พันบรรทัด)
- **EAF (Effort Adjustment Factor)**: ผลคูณของตัวแปร Cost Drivers 15 ตัว

ตารางสัมประสิทธิ์สำหรับ Intermediate Model:

ประเภทโครงการ (Mode)	a	b
Organic	3.2	1.05
Semi-detached	3.0	1.12
Embedded	2.8	1.20



ตัวอย่างโจทย์การคำนวณ

สถานการณ์: บริษัทของนักศึกษา กำลังจะพัฒนาระบบ "เวชระเบียนโรงพยาบาล (Hospital Medical Records)" ซึ่งจัดเป็นโครงการประเภท **Semi-detached** (มีความซับซ้อนปานกลาง)

1. ขนาดโครงการ: ประเมินไว้ที่ 50,000 บรรทัด (50 KLOC)
2. สภาพแวดล้อมโครงการ (Cost Drivers):
 - **Required Reliability (RELY):** สูงมาก (High) เพราะข้อมูลคนไข้ห้ามผิดพลาด -> ค่าตัวคูณ = 1.15
 - **Analyst Capability (ACAP):** ทีมวิเคราะห์เก่งมาก (High) -> ค่าตัวคูณ = 0.86 (ช่วยลดแรงงาน)
 - **Software Tools (TOOL):** ใช้เครื่องมือพื้นฐานธรรมดา (Nominal) -> ค่าตัวคูณ = 1.00
 - **Complexity (CPLX):** ซับซ้อนสูง (High) -> ค่าตัวคูณ = 1.15
 - (สมมติว่าตัวแปรอื่นๆ อีก 11 ตัว เป็นค่าปกติ หรือ 1.00 ทั้งหมด)

ขั้นตอนที่ 1: คำนวณหา EAF (Effort Adjustment Factor)

นำค่าตัวคูณของ Cost Drivers ทั้งหมดมาคูณกัน:

$$EAF = 1.15 \times 0.86 \times 1.00 \times 1.15 \times (1.00\dots)$$

$$EAF \approx 1.137$$

ความหมาย: สภาพแวดล้อมของโครงการนี้ทำให้ต้องใช้แรงงานเพิ่มขึ้นจากปกติประมาณ **13.7%**
(เพราะ $EAF > 1$) เนื่องจากความต้องการความน่าเชื่อถือและความซับซ้อนของงาน แม้ทีมงานจะเก่งก็ตาม

ขั้นตอนที่ 2: คำนวณ Effort (แรงงานที่ต้องใช้)

ใช้สูตร $E = a \times (KLOC)^b \times EAF$ (Semi-detached: $a = 3.0, b = 1.12$)

1. คำนวณแบบปกติก่อน (Nominal Effort):

$$E_{nom} = 3.0 \times (50)^{1.12}$$

$$(50)^{1.12} \approx 79.9$$

$$E_{nom} = 3.0 \times 79.9 \approx 239.7 \text{ Person-Months}$$

2. คูณด้วย EAF:

$$E_{real} = 239.7 \times 1.137$$

$$E_{real} \approx 272.5 \text{ Person-Months}$$

สรุป: โครงการนี้ต้องใช้แรงงานประมาณ **272.5 คน-เดือน**

ขั้นตอนที่ 3: คำนวณระยะเวลา (Time) และจำนวนคน (Person)

ใช้สูตรเดียวกับ Basic COCOMO แต่ใช้ค่า Effort ใหม่ที่ได้รับ

($c = 2.5$, $d = 0.35$ สำหรับ Semi- detached)

1. หา Time:

$$T = 2.5 \times (Effort)^d$$

$$T = 2.5 \times (272.5)^{0.35}$$

$$(272.5)^{0.35} \approx 7.1$$

$$T = 2.5 \times 7.1 \approx 17.75 \text{ Months}$$

2. หา Person:

$$P = E/T$$

$$P = 272.5/17.75 \approx 15.35 \text{ People}$$

บทวิเคราะห์

ถ้าใช้ Basic COCOMO คำนวณโจทย์ข้อนี้ โดยไม่ใช้ Cost Drivers จะได้ Effort ประมาณ 239 PM แต่ถ้าใช้ Intermediate COCOMO พบว่า Effort เพิ่มขึ้นเป็น 272.5 PM

ดังนั้น การที่ระบบต้องการ High Reliability (ความน่าเชื่อถือสูง) และมี High Complexity (ความซับซ้อนสูง) ทำให้ต้นทุนโครงการเพิ่มขึ้นอย่างเห็นได้ชัด แม้เราจะมีทีมงานที่เก่ง (ACAP High) มาช่วยดึงกราฟลงแล้วก็ตาม

3. Detailed COCOMO

เป็น COCOMO ขั้นละเอียด คือการก้าวเข้าสู่ความเป็นมืออาชีพในการบริหารโครงการขนาดใหญ่ คือในขณะที่ Intermediate COCOMO มองภาพรวมของโครงการเป็นก้อนเดียว แต่ Detailed COCOMO มองความจริงที่ว่า "แต่ละเฟสของการทำงาน ใช้ทรัพยากรและได้รับผลกระทบจากปัจจัยต่าง ๆ ไม่เท่ากัน"

ลักษณะสำคัญของ Detailed COCOMO ได้แก่

- 1) Phase-Sensitive Effort Multipliers ค่าปรับปรุงต้นทุน (Cost Drivers) 15 ตัว จะมีค่าไม่เท่ากันในแต่ละเฟส ตัวอย่างเช่น ทีมวิเคราะห์ระบบที่ไม่เก่ง (Low Analyst Capability) จะส่งผลกระทบอย่างหนักในช่วง Design (งานซ้ำ, ผิดเยอะ) แต่ส่งผลกระทบน้อยกว่าในช่วง Coding (เพราะคนเขียนโค้ดอาจจะเป็นคนละคนกัน หรือแบบเสร็จไปแล้ว)
- 2) Three-Level Hierarchy การประเมินจะแบ่งย่อยลงไปถึงระดับ Module (โมดูลย่อย) Subsystem (ระบบย่อย) และ System (ระบบรวม)

สถานการณ์ (Scenario)

บริษัทกำลังพัฒนาระบบ "Inventory Management (Organic Mode)" ขนาด 20,000 บรรทัด (20 KLOC) ถ้า
โฟกัสที่ผลกระทบของปัจจัย Analyst Capability (ACAP) หรือความสามารถของนักวิเคราะห์ระบบ ซึ่งในที่นี้เรา
ประเมินว่า ต่ำ (Low)

สมมติการแบ่งสัดส่วนแรงงาน (Phase Distribution) ตามมาตรฐานทั่วไปของ Organic Mode คือ

- Product Design (DD): 16% ของแรงงานรวม
- Programming (Code): 62% ของแรงงานรวม

ตารางค่าตัวคูณ (สมมติจากตารางมาตรฐานของ Detailed COCOMO)

สำหรับปัจจัย ACAP = Low

- ตัวคูณสำหรับเฟส Design: 1.35 (กินแรงงานเพิ่ม 35% เพราะออกแบบยากถ้าคนไม่เก่ง)
- ตัวคูณสำหรับเฟส Code: 1.10 (กินแรงงานเพิ่มแค่ 10% เพราะผ่านการออกแบบมาแล้ว)

74.24 ทุกเฟส
Person-Months

1. คำนวณแรงงานรวมปกติ (Nominal Effort)

ใช้สูตร Intermediate COCOMO เป็นฐาน ($a = 3.2, b = 1.05$ สำหรับ Organic)

$$E_{nom} = 3.2 \times (20)^{1.05}$$

$$(20)^{1.05} \approx 23.2$$

$$E_{nom} = 3.2 \times 23.2 \approx 74.24 \text{ Person-Months}$$

นี่คือแรงงานรวมถ้าทุกอย่างเป็นปกติ (Nominal)

2. คำนวณแยกเฟสแบบ Detailed

เราจะถือว่าพอใช้ Detailed COCOMO แล้ว แรงงานที่ใช้จริงในแต่ละเฟสจะเป็นเท่าไร

ก. เฟสการออกแบบ (Product Design)

1. แรงงานปกติของเฟสนี้: $74.24 \times 16\% = 11.88 \text{ PM}$
2. ปรับด้วยค่าตัวคูณ (ACAP Low = 1.35):

$$E_{design} = 11.88 \times 1.35 = 16.04 \text{ Person-Months}$$

66.67 เฟส Design & Code
Person-Months

ข. เฟสการเขียนโค้ด (Programming)

1. แรงงานปกติของเฟสนี้: $74.24 \times 62\% = 46.03$ PM

2. ปรับด้วยค่าตัวคูณ (ACAP Low = 1.10):

$$E_{code} = 46.03 \times 1.10 = 50.63 \text{ Person-Months}$$

3. รวมผลลัพธ์ (Total Effort)

หากเราคิดเฉพาะ 2 เฟสนี้ แรงงานรวมที่ปรับแก้แล้วจะเป็น:

$$Total = 16.04 + 50.63 = 66.67 \text{ Person-Months}$$

(ในความเป็นจริงต้องทำครบทุกเฟส ทั้ง *Planning* และ *Integration/Testing* ด้วย)

2. การวิเคราะห์ฟังก์ชันพอยต์ (Function Point Analysis - FPA)

การวิเคราะห์ฟังก์ชันพอยต์ (FPA) ได้รับการพัฒนาโดย แอลัน เจ. อัลเบรคต์ (Allan J. Albrecht) ที่ IBM ในปี ค.ศ. 1979 เพื่อเป็นทางเลือกในการวัดขนาดของซอฟต์แวร์นอกเหนือจากการนับบรรทัดของโค้ด (Lines of Code - LOC) หลักการสำคัญของ FPA คือการวัดขนาดของซอฟต์แวร์จากมุมมองของผู้ใช้ โดยเน้นที่ฟังก์ชันการทำงานที่ระบบส่งมอบให้กับลูกค้าหรือผู้ใช้ปลายทาง ทำให้เป็นอิสระจากภาษาการเขียนโปรแกรมหรือเทคโนโลยีที่ใช้ในการพัฒนา

FPA เป็นเทคนิคการประมาณขนาดซอฟต์แวร์ โดยมีหลักการสำคัญคือการวัดขนาดของซอฟต์แวร์จาก "สิ่งที่ผู้ใช้งานมองเห็นและได้รับจากระบบ" หรือ "ฟังก์ชันการทำงานเชิงตรรกะ" ไม่ใช่จำนวนบรรทัดโค้ดจริง

FPA พิจารณา **5 องค์ประกอบหลัก**ที่จัดกลุ่มเป็น 2 ประเภท คือ Data Functions (ข้อมูล) และ Transactional Functions (การประมวลผล)

1) Data Functions ฟังก์ชันข้อมูล - นับข้อมูลที่ระบบต้องจัดการ

- **Internal Logical Files (ILFs)** กลุ่มข้อมูลที่จัดเก็บและบำรุงรักษาภายในระบบเอง เช่น ตารางหลักในฐานข้อมูล
- **External Interface Files (EIFs)** กลุ่มข้อมูลที่อ้างอิงและใช้งานจากระบบภายนอก เช่น ตารางในฐานข้อมูลของระบบอื่นที่ต้องอ่านมาใช้

2) Transactional Functions ฟังก์ชันการประมวลผล - นับการปฏิสัมพันธ์

- **External Inputs (EIs)** การรับข้อมูลเข้าสู่ระบบจากผู้ใช้ เช่น หน้าจอ Login, หน้าจอบันทึกข้อมูล
- **External Outputs (EOs)** การนำข้อมูลออกจากระบบที่ซับซ้อนกว่า Inquiry เช่น การสร้างรายงาน, การแจ้งเตือนที่คำนวณซับซ้อน
- **External Inquiries (EQs)** การสอบถามข้อมูลที่ง่าย ๆ และแสดงผลทันที เช่น หน้าจอแสดงผลยอดคงเหลือ

โจทย์สถานการณ์

บริษัทพัฒนาซอฟต์แวร์กำลังประเมินขนาดของ "ระบบจัดการการลาหยุดของพนักงาน (Leave Management System)" โดยการวิเคราะห์ความต้องการเบื้องต้นได้ผลดังนี้

องค์ประกอบ	จำนวนที่นับได้ (Count)
ILF (ข้อมูลพนักงาน, บันทึกการลา)	2
EIF (อ้างอิงตารางหน่วยงานจากระบบบัญชี)	1
EI (หน้าจอ Login, หน้าจอยื่นใบลา, หน้าจออนุมัติ)	3
EO (รายงานสรุปการลาประจำเดือน)	1
EQ (หน้าจอสอบถามยอดวันลาคงเหลือ)	1

ขั้นตอนที่ 1: การนับ Unadjusted Function Point (UFP)

นำจำนวนองค์ประกอบที่นับได้มาประเมินความซับซ้อน (Complexity) โดยใช้ตารางมาตรฐาน

ตารางประเมินความซับซ้อน (Simplified for Exercise)

องค์ประกอบ	ความซับซ้อนที่ประเมิน (Degree)	ตัวคูณ FP
ILF (2)	ปานกลาง (Medium)	10
EIF (1)	ปานกลาง (Medium)	7
EI (3)	ง่าย (Low)	3
EO (1)	ปานกลาง (Medium)	5
EQ (1)	ง่าย (Low)	3

ก. การคำนวณ UFP

องค์ประกอบ	Count	ตัวคูณ FP (Weight)	ผลรวม (UFP)
ILF	2	10	20
EIF	1	7	7
EI	3	3	9
EO	1	5	5
EQ	1	3	3
รวม UFP (Unadjusted Function Point):			44

ขั้นตอนที่ 2: การปรับปรุงด้วย Value Adjustment Factor (VAF)

FPA จะปรับ UFP ให้เป็น Adjusted Function Point (FP) โดยการพิจารณาปัจจัยทางด้านเทคนิคและสิ่งแวดล้อมรวม 14 ปัจจัย (General System Characteristics - GSC) เช่น ความต้องการด้าน Data Communication, ความซับซ้อนในการติดตั้ง, ความต้องการด้าน Performance

- แต่ละปัจจัยจะมีคะแนนตั้งแต่ 0 (ไม่มีผล) ถึง 5 (มีผลอย่างมาก)
- Total Degree of Influence (TDI): คือผลรวมคะแนนจาก 14 ปัจจัย
- VAF (Value Adjustment Factor)

$$VAF = 0.65 + (TDI / 100)$$

สมมติฐานสำหรับโจทย์: จากการประเมิน 14 ปัจจัย พบว่ามี TDI (Total Degree of Influence) รวมเป็น 40 คะแนน

ก. การคำนวณ VAF

$$VAF = 0.65 + (40/100) = 0.65 + 0.40 = 1.05$$

ข. การคำนวณ Adjusted Function Point (FP)

$$FP = UFP \times VAF$$

$$FP = 44 \times 1.05 \approx 46.2$$

✓ ข้อสรุป

ขนาดของ "ระบบจัดการการลาหยุด" นี้คือ **46.2 Function Points**

การนำไปใช้ในการบริหารโครงการ:

หลังจากได้ค่า FP แล้ว ผู้จัดการโครงการจะสามารถประมาณการต้นทุนได้อย่างแม่นยำขึ้น โดยใช้ค่า FP นี้แปลงเป็น Effort (คน-เดือน) ผ่านค่า Productivity (ผลผลิต) ของบริษัทตนเอง:

$$\text{Effort (Person-Months)} = FP / \text{Productivity (FP/Person-Month)}$$

- ตัวอย่าง: ถ้าบริษัทมี Productivity อยู่ที่ **5 FP/Person-Month**
- **Effort ที่ต้องการ:** $46.2 / 5 = 9.24$ Person-Months